

PROGRAMACION DE REDES

Informe Tecnico

Desarrollo de una Aplicacion con Redes Neuronales
sobre Datasets de Kaggle

Perceptron Multicapa (MLP) aplicado al Heart Disease Dataset (Kaggle)

Trabajo Practico	PSR-TP01-C2
Integrantes	Carolina Lobo / Sabrina De Marco
Fecha de entrega	2 de septiembre de 2026

2. Introduccion y Eleccion del Dataset

2.1 Origen de los datos

Se utilizo el **Heart Disease Dataset**, publicado en Kaggle por el usuario johnsmith88, disponible en el siguiente enlace:

<https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

2.2 Descripcion del problema

El dataset es una version extendida del clasico Cleveland Heart Disease Dataset y contiene registros clinicos de pacientes (edad, presion arterial, colesterol, resultados de electrocardiograma, frecuencia cardiaca maxima, entre otros). El archivo original tiene 1025 filas, de las cuales varias son duplicados producto de la union de distintas fuentes; luego de la limpieza el dataset de trabajo quedo compuesto por **302 pacientes unicos** y 13 atributos de entrada.

El objetivo es **predecir la variable target**: si un paciente presenta (1) o no (0) riesgo de enfermedad cardiaca, a partir de sus indicadores clinicos. Se trata, por lo tanto, de un problema de **clasificacion binaria supervisada**.

Atributo	Descripcion
age	Edad del paciente (anios)
sex	Sexo (1 = hombre, 0 = mujer)
cp	Tipo de dolor de pecho (0-3)
trestbps	Presion arterial en reposo (mm Hg)
chol	Colesterol serico (mg/dl)
fbs	Glucemia en ayunas > 120 mg/dl (1 = si)
restecg	Resultado del electrocardiograma en reposo (0-2)
thalach	Frecuencia cardiaca maxima alcanzada
exang	Angina inducida por ejercicio (1 = si)
oldpeak	Depresion del segmento ST inducida por ejercicio
slope	Pendiente del segmento ST en el pico de ejercicio (0-2)
ca	Numero de vasos principales coloreados por fluoroscopia (0-4)
thal	Talasemia (0-3)
target	Variable objetivo: 1 = con enfermedad, 0 = sin enfermedad

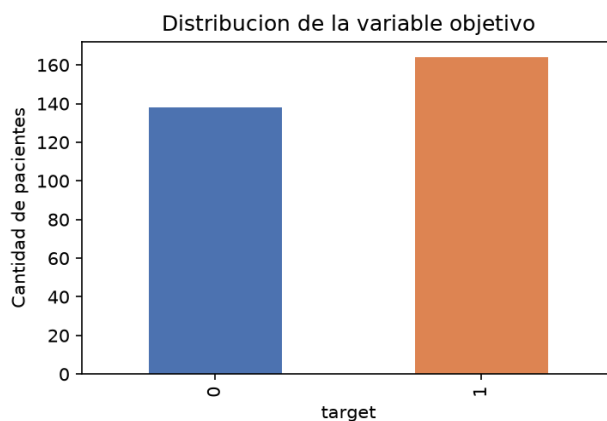


Figura 1. Distribucion de pacientes con y sin enfermedad cardiaca (302 registros).

3. Preprocesamiento y Normalizacion

El preprocesamiento se implemento en la clase `GestorDatos` del archivo `app_red_neuronal.py`, y comprende:

- **Limpieza:** eliminacion de filas duplicadas y de registros con valores faltantes (de 1025 a 302 registros unicos).
- **Separacion de variables:** matriz de atributos (X, 13 columnas) y variable objetivo (y = target).
- **Normalizacion (Standardizacion):** se aplico `StandardScaler` de `scikit-learn` sobre las 13 features.
- **Particion train/test (70/30):** ver seccion 4.2.

3.1 Metodo de normalizacion utilizado

Se utilizo **estandarizacion (Z-score)**, que transforma cada atributo para que tenga media 0 y desvio estandar 1. La formula aplicada a cada valor x de un atributo es:

$$z = (x - \mu) / \sigma$$

donde x es el valor original del atributo, μ (mu) es la media del atributo calculada sobre el conjunto de entrenamiento, y σ (sigma) es su desvio estandar, tambien calculado sobre el conjunto de entrenamiento. El escalador se ajusta (`fit`) unicamente con los datos de entrenamiento y luego se aplica (`transform`) tanto a train como a test, para evitar fuga de informacion (`data leakage`) del conjunto de prueba hacia el ajuste del escalador.

3.2 Justificacion de la normalizacion

La normalizacion es necesaria porque los atributos del dataset tienen escalas muy distintas: por ejemplo, `cho1` (colesterol) toma valores entre ~120 y ~570, mientras que `oldpeak` toma valores entre 0 y 6.5, o `sex` es binaria (0/1). Sin normalizar, los atributos de mayor magnitud dominarian el calculo de los pesos sinapticos durante el entrenamiento, ralentizando o incluso impidiendo la convergencia del descenso de gradiente. Al estandarizar todas las variables a una escala comparable, la red converge de forma mas rapida y estable, y ningun atributo recibe mas peso solo por tener una magnitud numerica mayor.

3.3 Comparacion: datos crudos vs. normalizados

A modo de verificacion, se muestran los primeros registros del dataset antes y despues de la normalizacion (la aplicacion permite ver esta misma comparacion de forma interactiva en la

pestanía 'Vista de Datos'):

Datos crudos (extracto)

age	sex	cp	trestbps	chol	thalach	oldpeak
52	1	0	125	212	168	1.0
53	1	0	140	203	155	3.1
70	1	0	145	174	125	2.6

Datos normalizados (mismas filas, StandardScaler)

age	sex	cp	trestbps	chol	thalach	oldpeak
-0.268	0.683	-0.935	-0.377	-0.668	0.806	-0.037
-0.157	0.683	-0.935	0.479	-0.842	0.238	1.774
1.725	0.683	-0.935	0.764	-1.403	-1.075	1.343

4. Diseño de la Red Neuronal y Parametros

4.1 Arquitectura

Se implementó un **Perceptron Multicapa (MLP)** mediante `MLPClassifier` de `scikit-learn`, encapsulado en la clase `RedNeuronalMLP`. Es una red feedforward totalmente conectada, entrenada por backpropagation, con la siguiente arquitectura:

Capa	Neuronas	Funcion de activacion
Entrada	13 (una por atributo)	-
Oculto 1	16	ReLU
Oculto 2	8	ReLU
Salida	1 (probabilidad de clase 1)	Sigmoide (softmax binaria)

4.2 Justificacion de la division 70/30

El dataset se dividió automáticamente en 70% para entrenamiento (211 pacientes) y 30% para prueba (91 pacientes), utilizando muestreo estratificado para conservar la proporción de clases en ambos subconjuntos. Se eligió esta proporción porque, tratándose de un dataset relativamente pequeño (302 registros), un 70/30 ofrece un equilibrio razonable entre disponer de suficientes ejemplos para que la red ajuste sus pesos correctamente, y reservar una porción de prueba lo bastante grande (30%, 91 pacientes) como para que las métricas de evaluación sean estadísticamente representativas y no estén sujetas a demasiada variabilidad por el tamaño de muestra.

4.3 Hiperparametros configurados

Hiperparametro	Valor
Tasa de aprendizaje (learning rate)	0.001
Numero de epocas (epochs / max_iter)	500
Optimizador (solver)	Adam

Funcion de perdida	Binary Cross-Entropy (log-loss)
--------------------	---------------------------------

Estos valores son ajustables de forma interactiva desde la aplicacion (pestanía 'Entrenamiento y Convergencia'), lo que permite experimentar con distintas configuraciones y observar su efecto sobre la convergencia y el desempenio del modelo.

5. Analisis de Resultados y Graficas

5.1 Analisis del error (convergencia)

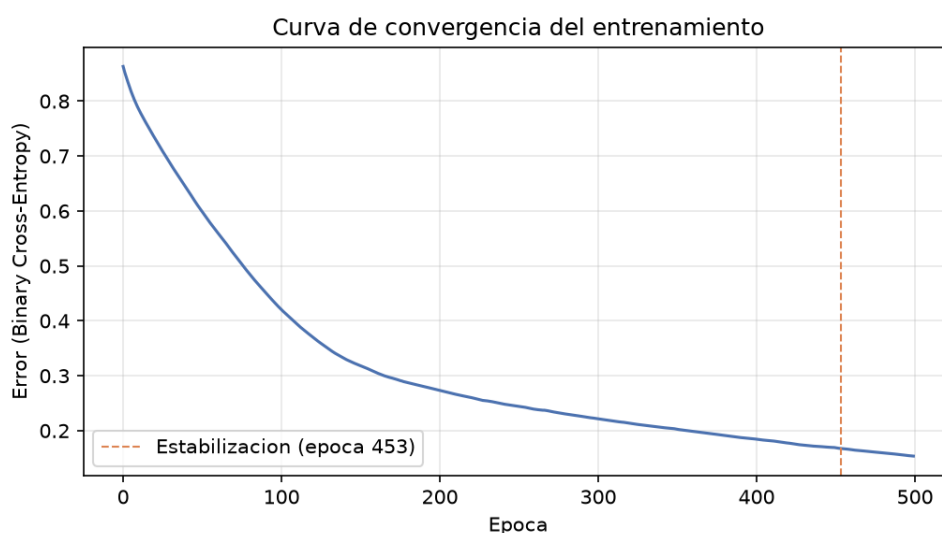


Figura 2. Evolucion del error (Binary Cross-Entropy) durante las 500 epocas de entrenamiento.

El error parte de 0.863 en la primera epoca y desciende de forma pronunciada durante las primeras ~100 epocas, para luego continuar descendiendo de forma mas lenta y gradual. El error final alcanzado es de 0.154. La curva se mantiene dentro de una banda estable (variacion menor al 2% respecto del valor final) recién a partir de la **epoca 453**, es decir, muy cerca del limite de 500 epocas configurado. Esto indica que el modelo **no llega a converger completamente** dentro de las 500 epocas: scikit-learn emite una advertencia de convergencia (ConvergenceWarning) confirmando que el optimizador aun podria seguir reduciendo el error si se entrenara por mas tiempo.

Se realizo una prueba adicional aumentando el limite a 800 epocas: el error de entrenamiento continuo bajando (hasta 0.075), pero el accuracy sobre el conjunto de test **empeoro** (de 0.813 a 0.769). Esto es un indicio de sobreajuste (overfitting): al entrenar por mas tiempo sobre un conjunto de entrenamiento pequeno (211 pacientes), la red comienza a memorizar particularidades del set de entrenamiento en lugar de generalizar, perdiendo capacidad predictiva sobre datos nuevos. Por este motivo se mantuvo max_iter = 500 como configuracion final.

5.2 Analisis de la prediccion (30% de prueba)

Metrica	Valor
Accuracy	0.813
Precision	0.848
Recall	0.796
F1-score	0.821

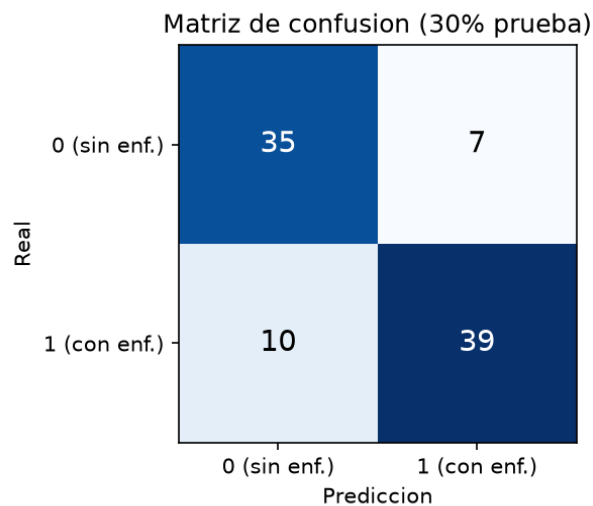


Figura 3. Matriz de confusion sobre el conjunto de prueba (91 pacientes, 30% del dataset).

El modelo alcanza una exactitud del 81.3% sobre el 30% de prueba. De los 91 pacientes evaluados, clasifíco correctamente a 74 (35 verdaderos negativos y 39 verdaderos positivos), cometiendo 7 falsos positivos y 10 falsos negativos. La precisión (0.848) indica que, de los pacientes que el modelo marco como positivos, un 84.8% realmente tenían la enfermedad; el recall (0.796) indica que el modelo detectó al 79.6% de los casos positivos reales. En un contexto clínico, el recall es especialmente relevante porque un falso negativo (no detectar una enfermedad presente) tiene consecuencias más graves que un falso positivo.

6. Conclusiones

El Perceptron Multicapa logró aprender un patrón útil a partir de los indicadores clínicos del dataset, alcanzando un F1-score de 0.821 sobre datos nunca vistos durante el entrenamiento, lo cual confirma que la red generaliza razonablemente bien y no se limita a memorizar el conjunto de entrenamiento.

Respecto de la convergencia, el modelo no se estabiliza por completo dentro de las 500 épocas configuradas (la curva de error sigue con una leve tendencia descendente hasta el final). Sin embargo, el experimento con 800 épocas mostró que forzar una convergencia más completa del error de entrenamiento no mejora el desempeño real del modelo, sino que lo empeora por sobreajuste. Esto evidencia que, en este problema, el número de épocas no debería elegirse únicamente en función de minimizar el error de entrenamiento, sino monitoreando el desempeño sobre el conjunto de prueba.

Como mejoras futuras se propone: (1) incorporar validación cruzada (k-fold) para obtener una estimación más robusta del desempeño, menos dependiente de una única partición 70/30; (2) utilizar la opción de `early_stopping` de scikit-learn para detener el entrenamiento automáticamente cuando el desempeño en un conjunto de validación deja de mejorar, evitando el sobreajuste observado; (3) ampliar el dataset con más registros, dado que el tamaño reducido (302 pacientes) limita la capacidad de la red para generalizar con mayor precisión.

Anexo. Estructura del proyecto entregado

dataset.csv - Heart Disease Dataset descargado de Kaggle
app_red_neuronal.py - código fuente en P00 (clases GestorDatos, RedNeuronalMLP,
DashboardApp)
Informe_Tecnico_TP.pdf - este documento